

Теория автоматов и формальных языков

Лекция 1

ОСНОВНЫЕ ПОНЯТИЯ ТЕОРИИ АЛГОРИТМОВ

к.т.н., доц.

Буряченко В.В.

2024



Понятие алгоритма

Интуитивное определение:

- **Алгоритм** - это строгая и четкая конечная система правил решения некоторого класса задач, определяющая процесс преобразования исходных данных в искомый результат.



Основные требования к алгоритмам

- Каждый алгоритм имеет дело с данными - входными, промежуточными и выходными.
- Данные для своего размещения требуют памяти.

Свойства алгоритма:

1. **Дискретность.**
2. **Определенность** (или детерминированность).
3. **Результативность** (или конечность).
4. **Массовость.**



Основные требования к алгоритмам

Для задания алгоритма необходимо выделить и описать следующие его элементы:

- **набор объектов, составляющих совокупность данных: исходных, промежуточных и конечных;**
- **правило начала;**
- **правила непосредственной переработки информации;**
- **правило окончания;**
- **правило извлечения результатов.**



Математическое определение алгоритма

Математическое определение алгоритма необходимо по следующим причинам:

1. только при наличии формального определения алгоритма можно сделать вывод о разрешимости или неразрешимости каких-либо проблем;
 2. это дает возможность для сравнения алгоритмов, предназначенных для решения одинаковых задач;
 3. это дает возможность для сравнения различных проблем по сложности алгоритмов их решения.
-
- **Алгоритмами в современной математике принято называть конструктивно задаваемые соответствия между изображениями объектов в абстрактных алфавитах.**

объектом алгоритма может оказаться все, что угодно



Абстрактный алфавит

- **Абстрактным алфавитом** называется любая конечная совокупность объектов, называемых буквами или символами данного алфавита.
- Основным требованием к алфавиту является его конечность.



Математическое определение алгоритма

- **Алгоритм** есть четкая конечная система правил для преобразования слов из некоторого алфавита в слова из этого же алфавита.
 - Слово, к которому применяется алгоритм, называется **входным**.
 - Слово, вырабатываемое в результате применения алгоритма, называется **выходным**.
 - Совокупность слов, к которым применим данный алгоритм, называется **областью применимости** этого алгоритма.

Алфавит. Цепочка

Алфавит — конечное множество символов (или букв).

Цепочка (слово или строка) в алфавите (над алфавитом) Σ — конечная последовательность записанных подряд символов алфавита Σ (в том числе последовательность, не содержащая ни одного символа).

Цепочка, не содержащая ни одного символа, называется **пустой** и обозначается через ε .

- Пример: ε , a , ba , $aaaba$ — цепочки в алфавите $\{a, b\}$.

Цепочка, состоящая из $i = 0, 1, 2, \dots$ символов a , обозначается через a^i . Например, $a^0 = \varepsilon$, $a^1 = a$, $a^3 = aaa$.

- Число символов (не обязательно различных) в цепочке w называется длиной этой цепочки и обозначается $|w|$.

Конкатенация. Вхождение

Пусть u, v, w — цепочки (в некотором алфавите).

- Конкатенацией цепочек u и v называется цепочка uv .
- Примеры: если $u = ab$ и $v = cd$, то $uv = abcd$;
для любой цепочки x имеем $\varepsilon x = x\varepsilon = x$.
- Цепочка u называется **префиксом** (или началом) цепочки uv .
- Цепочка v называется **суффиксом** (или концом) цепочки uv .

Префикс (соответственно, суффикс) цепочки w называется **собственным**, если он отличен от ε и от w .

- Цепочка v входит в цепочку (называется **подцепочкой** цепочки) uvw .

Пример: **ab** — собственный префикс и подцепочка цепочки **abb**.

Определение формального языка

Пусть Σ — алфавит.

- Множество **всех цепочек** в алфавите Σ обозначается через Σ^* .
- Множество **всех непустых** цепочек в алфавите Σ обозначается через Σ^+ .
- Языком (**формальным языком**) в алфавите (над алфавитом) Σ называется какое угодно подмножество множества Σ^* .

Пример языка: язык описаний типов LD

Алфавит языка LD: $\{i, :, ;, \text{int}, \text{real}\}$.

- Каждая цепочка языка LD имеет вид:
 - непустой список букв i , разделённых точкой с запятой,
 - затем двоеточие,
 - один из символов int или real .

Примеры:

- $i : \text{real} \in \text{LD}$,
- $i;i;i : \text{int} \in \text{LD}$.

Пример языка: язык арифметических выражений LA

Алфавит языка LA : $\{x, +, *, (,)\}$.

- Индуктивное определение языка LA :
 - $x \in LA$;
 - если $u \in LA$ и $v \in LA$, то $(u) \in LA$, $u + v \in LA$ и $u * v \in LA$.
- Примеры: (x) , $x + x * x$, $(x + x) * x \in LA$.



Алгоритмические модели

- Выделяют **три основных типа универсальных алгоритмических моделей**, различающихся исходными эвристическими соображениями относительно того, что такое алгоритм.
- Первый тип связывает понятие алгоритма с наиболее традиционными понятиями математики - **вычислениями и числовыми функциями**.
- Второй тип основан на представлении алгоритма **как некоторого детерминированного устройства**, способного выполнять в каждый отдельный момент некоторые примитивные операции, или инструкции.
- Третий тип алгоритмических моделей - **это преобразования слов в произвольных алфавитах**, в которых элементарными операциями являются подстановки, т.е. замены части слова (подслова) другим словом.



Понятие алгоритма в современной математике: рекурсивные функции

- **Рекурсивные функции** (также известные как примитивно-рекурсивные и общерекурсивные функции) предложены в рамках формальной теории вычислений для описания алгоритмов. Эти функции описывают вычислимые процессы как системы, где новые значения могут вычисляться на основе предыдущих шагов.

Примитивно-рекурсивные функции:

- Примитивно-рекурсивные функции определяются на основе базовых функций (нулевой функции, функции преемника и проекций), а затем строятся с использованием операций композиции и рекурсии.
- Примером может быть функция сложения, определяемая рекурсией на основе сложения с единицей.

Вклад рекурсивных функций в понятие алгоритма

- Рекурсивные функции предоставили математическую основу для определения вычислимых функций.
- Позволили классифицировать задачи по их вычислимости и сложности.

Пример: Рассмотрим функцию Фибоначчи, которая может быть определена рекурсивно:

$$F(0) = 0$$

$$F(1) = 1$$

$$F(n) = F(n - 1) + F(n - 2) \quad \text{для } n > 1$$

Общерекурсивные функции

- **Общерекурсивные функции** расширяют понятие примитивно-рекурсивных функций, вводя оператор **минимизации**, который позволяет искать минимальное значение, удовлетворяющее определенному условию. Это расширение позволяет выразить алгоритмы, которые не обязательно завершаются за конечное время, как в случае некоторых задач поиска.
- Эта модель использовалась для доказательства тезиса Чёрча, который утверждает, что любая вычислимая функция может быть вычислена с помощью общерекурсивных функций.

Применение:

- Рекурсивные функции позволяют строго формализовать понятие вычислимости и являются ключевым компонентом теории вычислений. Они особенно полезны для описания алгоритмов, где результат вычисления зависит от предыдущих шагов (как в случае факториала или чисел Фибоначчи).



Формализация понятия алгоритма при помощи машины Тьюринга

- **Машина Тьюринга** — одна из самых фундаментальных моделей вычислений, предложенная Аланом Тьюрингом в 1936 году. Она описывает алгоритмы как последовательности элементарных операций на бесконечной ленте данных.

Описание модели:

- Машина Тьюринга состоит из бесконечной ленты, разделённой на ячейки, и головки, которая может считывать символы с ленты, записывать новые символы, а также двигаться по ленте влево или вправо.
- Алгоритм работы машины Тьюринга задается конечным набором состояний и правил перехода, которые определяют, что делает машина при встрече с тем или иным символом в определенном состоянии.



Формализация понятия алгоритма при помощи машины Тьюринга

Основные характеристики:

- Машина Тьюринга может моделировать **любой алгоритм**, который можно представить как вычислимую функцию. Она способна вычислять любые функции, которые описываются другими подходами, такими как рекурсивные функции или лямбда-исчисление.
- Важное свойство машины Тьюринга — **универсальность**. Универсальная машина Тьюринга может имитировать любую другую машину Тьюринга, что сделало эту модель основной для понимания универсальных компьютеров.

Применение:

- Машина Тьюринга — ключевая модель для теоретической информатики. Она используется для изучения вопросов **вычислимости** (какие задачи могут быть решены с помощью алгоритмов) и **сложности** (сколько времени или памяти потребуется для их решения).

Формализация понятия алгоритма через формальные грамматики

- **Формальные грамматики** — это подход, развившийся в теории языков и лингвистике, предложенный Ноамом Хомским в 1950-х годах. Они описывают алгоритмы через правила порождения символов и строк, что делает их полезными для моделирования синтаксической структуры языков программирования и естественных языков.
- **Контекстно-свободные грамматики (CFG)** описывают правила для порождения строк в языке. Каждое правило в такой грамматике позволяет заменить один символ (или нетерминал) на комбинацию терминалов и нетерминалов. Эти грамматики используются для описания языков программирования и **синтаксического анализа**.
- Алгоритм можно рассматривать как процесс, который преобразует начальный символ (или строку) по правилам грамматики, последовательно порождая новые строки, пока не будет достигнуто конечное состояние (строка), являющееся результатом вычисления.

Формализация понятия алгоритма через формальные грамматики

- Пример грамматики арифметических выражений:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T \times F \mid T \div F \mid F$$

$$F \rightarrow (E) \mid \text{number}$$

Применение:

- Формальные грамматики широко применяются в теории языков программирования, компиляции и лингвистике. Они также полезны для описания синтаксических структур алгоритмов и процессов.



Сложность алгоритма

- Сложность алгоритма измеряется в зависимости от размера входных данных и выражается в виде функции, показывающей количество шагов (временная сложность) или объем используемой памяти (пространственная сложность).

Асимптотическая нотация:

- $O(1)$ — константная сложность. Алгоритм выполняется за фиксированное время, независимо от размера входных данных.
- $O(n)$ — линейная сложность. Время выполнения алгоритма растет линейно с увеличением размера входных данных.
- $O(\log n)$ — логарифмическая сложность. Алгоритм выполняется за логарифмическое время по размеру входных данных.
- $O(n^2)$ — квадратичная сложность. Время выполнения алгоритма растет пропорционально квадрату размера входных данных.



Алфавитный оператор

- **Алфавитный оператор** — это понятие, которое используется в теории формальных языков, теории автоматов и компьютерной лингвистике. Оно относится к операциям, которые производятся над символами или строками, составляющими "алфавит" формального языка.
- **Алфавит** — это конечное множество символов, которые используются для построения строк или слов в формальном языке. Например, в бинарной системе алфавит состоит из символов $\{0, 1\}$, а в английском языке алфавитом будет множество букв $\{A, B, C, \dots, Z\}$.
- **Алфавитный оператор** — это операция, которая определена на элементах этого алфавита или на строках (последовательностях символов), составленных из символов алфавита.

Примеры алфавитных операторов

1. **Конкатенация:** Если Σ — это алфавит, $a, b \in \Sigma$, то конкатенация — это операция объединения двух символов в одну строку. Например, для алфавита $\Sigma = \{a, b, c\}$, конкатенация a и b даст строку ab .
2. **Реверсирование строки:** Если дана строка w , составленная из символов алфавита, то реверсирование строки w — это операция, которая меняет порядок символов на противоположный. Например, если $w = abc$, то реверсирование даст строку cba .
3. **Удаление символов:** Операция, при которой из строки удаляется определённый символ или набор символов. Например, удаление всех символов 'a' из строки 'banana' даст строку 'bnn'.
4. **Замена символов:** Операция, при которой определённый символ или набор символов заменяется другим символом или строкой. Например, замена всех 'a' на 'e' в строке 'banana' даст строку 'benene'.



Применение алфавитных операторов

Алфавитные операторы важны в различных областях информатики и лингвистики:

- **Теория автоматов:** Операции над алфавитами используются при проектировании и анализе автоматов, которые принимают или отвергают строки.
- **Компиляция:** При анализе и синтаксическом разборе программ исходный код рассматривается как строки символов из определенного алфавита, и различные алфавитные операторы используются для обработки этого кода.
- **Криптография:** В некоторых шифровальных алгоритмах используются операции над символами алфавита для шифрования и расшифровки сообщений.



Эквивалентные алгоритмы

- **Два алгоритма считаются равными**, если равны соответствующие им алфавитные операторы и совпадает система правил, задающих действие этих алгоритмов на выходные слова.
- **Два алгоритма считаются эквивалентными**, если у них совпадают алфавитные операторы, но не совпадают способы их задания (система правил).



Лабораторная работа №1 «Математическое понятие алгоритма»

- Во всех заданиях необходимо разработать **блок-схемы алгоритмов** и проанализировать процесс реализации алгоритма, т.е. представить на практике **последовательность шагов**, которая будет порождена при применении алгоритма к **конкретным исходным данным**.
- Привести **математическое описание алфавита** для реализуемого алгоритма.
- Привести пример набора **исходных данных** для алгоритма.